LABORATORY OBSERVATION / RECORD

Course Full Stack Web Development
Course Code 23CM4121
Experiment No. 6
Student Name P. Govindamma
Roll No. A24126552104
Date 22-09-2026

1. TITLE

Implement HTTP Methods (GET, POST, PUT and DELETE) using JSON Server

2. AIM

To implement the HTTP methods GET, POST, PUT and DELETE on a student record resource using JSON Server as a fake REST API and the fetch() function of JavaScript.

3. OBJECTIVE

The objectives of this experiment are:

- To set up JSON Server and create a REST API from a db.json file.
- To read all student records using the GET method.
- To add a new student record using the POST method.
- To update an existing student record using the PUT method.
- To remove a student record using the DELETE method.
- To use fetch() with promises (.then and .catch) to call the API and display the result in the console.

4. THEORY

JSON Server is a tool that creates a complete fake REST API from a single JSON file without writing any server code. Each top-level array in db.json (here, students) becomes a resource with its own URL, for example http://localhost:3000/students.

HTTP methods: GET reads data from the server, POST sends new data to be created, PUT replaces an existing record with the data sent, and DELETE removes a record. A record is identified by adding its id to the URL, for example /students/1.

fetch() API: fetch(url, options) is a built-in JavaScript function used to send HTTP requests. The options object contains the method, the headers (for example Content-Type: application/json) and the body. JSON.stringify() converts a JavaScript object into a JSON string for the body.

Promises: fetch() returns a promise. The first .then() converts the response into JSON using response.json(), the next .then() uses the data, and .catch() handles any error such as the server not running.

Status: response.ok is true when the server returns a status in the range 200 to 299, which is used in the DELETE request to confirm the deletion.

5. ALGORITHM / PROCEDURE

- Create a project folder and install JSON Server using npm install json-server.
- Create db.json with a students array that contains the sample records (id, name, branch and marks).
- Start the server using npx json-server db.json; it runs at http://localhost:3000.
- Create get.js that calls fetch() on /students and prints the data using console.log().
- Create post.js that sends a POST request with the JSON body for a new student (Anu, CSE, 95).
- Create put.js that sends a PUT request to /students/1 with the complete updated record.
- Create delete.js that sends a DELETE request to /students/3 and prints a message if response.ok is true.
- Keep the server running and execute each file in another terminal using node get.js, node post.js, node put.js and node delete.js.
- Run get.js again to verify the changes and compare the actual output with the expected output.

6. PROGRAM

db.json

```
{
  "students": [
    { "id": "1", "name": "Ravi", "branch": "CSE", "marks": 88 },
    { "id": "2", "name": "Sita", "branch": "ECE", "marks": 92 },
    { "id": "3", "name": "Kiran", "branch": "IT", "marks": 79 }
  ]
}
```

get.js

```
fetch("http://localhost:3000/students")
  .then(response => response.json())
  .then(data => console.log(data))
  .catch(error => console.log(error));
```

post.js

```
fetch("http://localhost:3000/students", {
  method: "POST",
  headers: {
    "Content-Type": "application/json"
  },
  body: JSON.stringify({
    name: "Anu",
    branch: "CSE",
    marks: 95
  })
})
  .then(response => response.json())
  .then(data => console.log(data))
  .catch(error => console.log(error));
```

put.js

```
fetch("http://localhost:3000/students/1", {
```

```
  method: "PUT",
  headers: {
    "Content-Type": "application/json"
  },
  body: JSON.stringify({
    id: "1",
    name: "Ravi Kumar",
    branch: "CSE",
    marks: 95
  })
})
  .then(response => response.json())
  .then(data => console.log(data))
  .catch(error => console.log(error));
```

delete.js

```
fetch("http://localhost:3000/students/3", {
  method: "DELETE"
})
  .then(response => {
    if (response.ok) {
      console.log("Student deleted successfully");
    }
  })
  .catch(error => console.log(error));
```

7. CODE EXPLANATION

Code	Explanation
db.json	Database file; the students array becomes the /students resource.
fetch(url)	Sends an HTTP request to the given URL (GET by default).
response.json()	Converts the response body into a JavaScript object.
.then(data => console.log(data))	Prints the data received from the server in the console.
.catch(error => ...)	Handles errors such as the server not running.
method: "POST"	Sends a request that creates a new record in /students.
method: "PUT"	Replaces the record with the given id by the data sent.
method: "DELETE"	Removes the record with the given id from /students.
headers: { "Content-Type": "application/json" }	Tells the server that the request body is in JSON format.
JSON.stringify({...})	Converts the JavaScript object into a JSON string for the body.
/students/1, /students/3	The id in the URL selects the record to update or delete.
response.ok	True when the status code is between 200 and 299.

8. TEST CASES AND EXECUTION DATA

Each HTTP method was executed and the actual output was recorded against the expected output.

Test Case	Route	Action	Expected Result	Actual Result	Status
TC-01	Server	Run npx json-server db.json	Server starts on port 3000	Server running at http://localhost:3000	Pass
TC-02	GET /students	Run node get.js	All student records returned	Array of 3 students printed	Pass
TC-03	POST /students	Run node post.js	New student Anu added with an id	Record for Anu returned with id 4	Pass
TC-04	PUT /students/1	Run node put.js	Record 1 replaced by Ravi Kumar, marks 95	Updated record printed	Pass
TC-05	DELETE /students/3	Run node delete.js	Student 3 removed	Student deleted successfully	Pass
TC-06	GET /students	Run node get.js again	List shows all the changes	Ravi Kumar, Sita and Anu listed	Pass

9. OUTPUT

JSON Server was kept running in one terminal and each file was executed with Node.js in another terminal. The output for each test case is given below.

Output 1 (TC-01): Server start

```
$ npx json-server db.json
JSON Server started on PORT :3000
http://localhost:3000/students
```

Output 2 (TC-02): GET

```
$ node get.js
[
  { id: '1', name: 'Ravi', branch: 'CSE', marks: 88 },
  { id: '2', name: 'Sita', branch: 'ECE', marks: 92 },
  { id: '3', name: 'Kiran', branch: 'IT', marks: 79 }
]
```

Output 3 (TC-03): POST

```
$ node post.js
{ id: '4', name: 'Anu', branch: 'CSE', marks: 95 }
```

Output 4 (TC-04): PUT

```
$ node put.js
{ id: '1', name: 'Ravi Kumar', branch: 'CSE', marks: 95 }
```

Output 5 (TC-05): DELETE

```
$ node delete.js
Student deleted successfully
```

Output 6 (TC-06): GET after all operations

```
$ node get.js
[
```

```
  { id: '1', name: 'Ravi Kumar', branch: 'CSE', marks: 95 },
  { id: '2', name: 'Sita', branch: 'ECE', marks: 92 },
  { id: '4', name: 'Anu', branch: 'CSE', marks: 95 }
]
```

10. RESULT

Thus, the HTTP methods GET, POST, PUT and DELETE were successfully implemented using JSON Server and the fetch() function. Student records were read, added, updated and deleted, and all test cases were executed and passed.